

Design Patterns

Luca Didaci

Table of contents

1	DESIGN PATTERNS	1
1.1	DESIGN PATTERNS	1
1.2	Categories of patterns	2
1.3	Idioms - STRING COPY	2
1.4	Idioms - SIZEOF TO VARIABLES	3
1.5	Inspiring principles of design patterns	3
1.6	The Open-Closed Principle	3

1 DESIGN PATTERNS

1.1 DESIGN PATTERNS

“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

Christopher Alexander, “A Pattern Language”, 1977. https://en.wikipedia.org/wiki/Christopher_Alexander

The idea was adopted by software engineering **Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (GoF) - Design Patterns: Elements of Reusable Object-Oriented Software (1994)**

A pattern has four essential elements:

1. The **pattern name** is a handle we can use to describe a design problem, its solutions, and consequences in a word or two. It makes it easier to think about designs and to communicate them and their trade-offs to others.
2. The **problem** describes when to apply the pattern. It explains the problem and its context.

-
3. The **solution** describes the elements that make up the design, their relationships, responsibilities, and collaborations. **The solution doesn't describe a particular concrete design or implementation**, because a pattern is like a template that can be applied in many different situations. Instead, the pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects in our case) solves it.
 4. The **consequences** are the results and trade-offs of applying the pattern.

1.2 Categories of patterns

- Architectural patterns
- Design patterns
- Language level patterns. This is the lowest level of the pattern-categories, also known as idioms

1.3 Idioms - STRING COPY

Naive implementation of string copy

```
void strcpy(char *dest, char *source){
    while ((*source)!='\0'){
        *dest=*source;
        source++;
        dest++;
    }
    *dest='\0';
}
```

The implementation you can find in standard libraries or in Kernighan & Ritchie

```
void strcpy(char *dest, char *source){
    while(*dest++ = *source++);
}
```

STRING COPY - real implementations

<https://italiancoders.it/per-qualche-strlen-in-piu/>

“invece di effettuare un semplice test a 0 un byte alla volta, effettuano il test su 4 bytes alla volta [...] questo sistema moltiplica la velocità di ricerca del fine stringa, permettendo un notevole risparmio di tempo per stringhe molto grandi.”

1.4 Idioms - SIZEOF TO VARIABLES

```
HelloTelegram *p_telegram = malloc( sizeof( HelloTelegram ));  
  
HelloTelegram *p_telegram = malloc(sizeof( *p_telegram ));
```

1.5 Inspiring principles of design patterns

- Abstraction
- Generality
- Anticipation of change
- Single-responsibility (separation of concerns)
- Modularity
- Reuse
- Don't Repeat Yourself (DRY)
- Open-Closed Principle
- Low coupling

(Single-responsibility principle and open-closed principle are the first 2 of the so-called **SOLID** principles)

1.6 The Open-Closed Principle

“Software entities should be **open for extension**, but **closed for modification**”

[Martin. “Agile Software Development: Principles, Patterns, and Practices”, 2002]

This principle states that software components — such as classes, modules, and functions — should allow their behavior to be extended **without modifying existing code**.

How? By **adding new code** (e.g., new functions or modules) **rather than altering the original implementation**. This reduces the risk of introducing bugs in tested code and promotes a more maintainable, loosely coupled design.

Note: It is nearly impossible to design a system where modules are completely closed to all types of changes. However, good design aims to minimize unnecessary modifications to existing code.

1 DESIGN PATTERNS

Example of **control coupling** AND **breach of the Open-Closed Principle**

(A detailed discussion follows in the next slides)

```
Bool isMultiple(int n, int d);}
Bool isGreaterThan(int n, int d);
void fizzBuzz(int i, int div1, int div2, int sw_f){// fizzbuzz function
    Bool fizz, buzz;
    switch (sw_f){
        case 1:
            fizz= isMultiple(i, div1);
            buzz= isMultiple(i, div2);
            break;
        case 2:
            fizz= isGreaterThan(i, div1);
            buzz= isGreaterThan(i, div2);
            break;
    }
    //...
}

int main(){
    int i, div1, div2, sw_f;

    i = 10;
    div1=3;
    div2=5;
    sw_f=1; //switch value: 1 or 2

    fizzBuzz( i, div1, div2, sw_f);
}
```
